



ARTIFICIAL NEURAL NETWORKS (ANN) LAB

13

Ms. Shakeela Shaheen

LAB 13: INTRODUCTION TO FINE-TUNING LARGE LANGUAGE MODELS (LLMS'S)

Learning Objectives

By the end of this lab, students will be able to:

- Understand what **fine-tuning** is at a deep conceptual and mathematical level
- Connect fine-tuning with **Transformers and Attention (Q, K, V)**
- Differentiate between **full fine-tuning, parameter-efficient methods, and prompting**
- Implement **practical fine-tuning pipelines using HuggingFace**
- Apply **LoRA, QLoRA, and adapter-based training**
- Understand **instruction tuning, domain adaptation, and alignment techniques**
- Evaluate and debug fine-tuned models
- Build a **real-world domain-specific LLM system**

Imagine, you are working at a startup building an **AI-powered academic assistant**.

You are using a pretrained model like:

- GPT-4
- Wav2Vec2

But:

It gives **generic answers**

It lacks **academic tone**

It doesn't follow structured explanations

Your goal: Fine-tune the model to behave like a **university-level teaching assistant (A SPECIFIC TASK)**

1. What is Fine-Tuning

Fine-tuning is:

Adjusting pretrained model weights using **task-specific data**

What's REALLY happening?

From previous lab:

$$Q = XW_q, K = XW_k, V = XW_v$$

These matrices encode **how the model understands relationships**

During fine-tuning:

- These weights are **updated slightly**
- The model **reshapes attention patterns**

Key Insight

You are not teaching language, you are **reshaping how attention flows**

2. Types of Model Adaptation:

(A) Full Fine-Tuning

Update all parameters

- ✓ Best performance
- + Very expensive
- + Needs large GPU

(B) Prompt Engineering (NO training)

Example:

Explain like a professor: What is recursion?

- ✓ No cost
- + Limited control

(C) Instruction Tuning

Train model on:

```
{  
  "instruction": "...",  
  "output": "..."  
}
```

This is how ChatGPT-like models are built

(D) Parameter Efficient Fine-Tuning (PEFT)

This is where modern AI lives.

3. LoRA (Low-Rank Adaptation)

Core Idea:

Instead of updating large weight matrix W :

$$W' = W + AB$$

Where:

- $A \in \mathbb{R}(d \times r)$
- $B \in \mathbb{R}(r \times d)$
- $r \ll d$

Interpretation:

- W = original knowledge
- AB = small adjustment

Applied to:

["q_proj", "k_proj", "v_proj"]

Meaning: You are directly modifying **attention behavior**

- ✓ Train faster
- ✓ Uses less memory
- ✓ Works on GPUs with low VRAM
- ✓ No need to store full model

4. QLoRA (NEXT LEVEL)

Problem: Even LoRA needs memory

Solution:

QLoRA = Quantization + LoRA

- Model weights stored in **4-bit precision**
- Training still high quality

✓ **Benefits:**

- Works on low GPUs
- Extremely efficient
- Industry standard

5. Adapters (Alternative to LoRA)

Idea:

Insert small neural layers between transformer layers

$$Output = Transformer(X) + Adapter(X)$$

✓ Modular

✓ Reusable

✓ Stackable

You already know a Transformer block looks like:

$$Input \rightarrow Attention \rightarrow Add \& Norm \rightarrow FFN \rightarrow Add \& Norm$$

Inside this pipeline, **adapters are inserted**.

Adapters are typically added:

- After Attention
- After Feedforward (FFN)

So the block becomes:

$$X \rightarrow Attention \rightarrow Adapter \rightarrow FFN \rightarrow Adapter$$

What is an Adapter (mathematically)?

An adapter is a **small neural network**:

$$Adapter(X) = W_{up}(\sigma(W_{down}(X)))$$

Component	Meaning
W_{down}	Reduces dimension
W_{up}	Expands back
σ	Activation (ReLU, GELU)

Why reduce dimension first?

Because we want **few parameters**

Example:

- Original dimension: 768
- Reduced to: 64

Massive reduction in trainable parameters

Input:

$X \rightarrow (\text{batch_size}, \text{seq_len}, 768)$

Step 1: Down Projection

$$H = XW_{down}$$

$(768 \rightarrow 64)$

Step 2: Activation

$$H = \sigma(H)$$

Step 3: Up Projection

$$\text{Adapter}(X) = HW_{up}$$

$(64 \rightarrow 768)$

4. Final Output (CRITICAL)

$$\text{Output} = X + \text{Adapter}(X)$$

THIS is the key idea: You are NOT replacing the transformer, you are **adding a correction**

Think of it like a **University Professor**

- Transformer = expert professor
- Adapter = teaching assistant

Professor already knows everything

BUT:

- TA adjusts explanation for:
 - beginners
 - domain-specific context
 - style

Adapter = **small adjustment layer**

6. RLHF (Reinforcement Learning from Human Feedback)

Used in:

- ChatGPT

Pipeline:

1. Pretraining
2. Fine-tuning
3. Human feedback
4. Reward model
5. Policy optimization

Why?

To align model with:

- Safety
- Helpfulness
- Tone

8. Data for Fine-Tuning

Types:

Type	Example
Instruction	“Explain X”
Conversational	Chat logs
Domain-specific	Medical/legal
Code	GitHub

Important:

Model is only as good as its data

9. FULL PRACTICAL PIPELINE

Step 1: Load Dataset

```
from datasets import load_dataset

dataset = load_dataset("tatsu-lab/alpaca")
```

Step 2: Tokenization

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("gpt2")
```

Step 3: Load Model

```
from transformers import AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained("gpt2")
```

Step 4: Apply LoRA

```
from peft import LoraConfig, get_peft_model

config = LoraConfig(
    r=8,
    lora_alpha=16,
    target_modules=["q_proj", "v_proj"]
)

model = get_peft_model(model, config)
```


Step 5: Training

```
from transformers import Trainer, TrainingArguments

training_args = TrainingArguments(
    output_dir="./results",
    num_train_epochs=3,
    per_device_train_batch_size=4
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=dataset["train"]
)

trainer.train()
```

What Happens During Training?

Loop:

1. Input → tokens
2. Model predicts next token
3. Compare with ground truth
4. Compute loss
5. Backpropagation

Best Practices

- ✓ Clean dataset
- ✓ Small learning rate
- ✓ Use validation
- ✓ Prefer LoRA / QLoRA
- ✓ Monitor outputs

LAB TASK

Fine-Tuning a Domain-Specific Academic Assistant (LLMs)

Objective

Design, fine-tune, and critically evaluate a domain-specific Large Language Model (LLM) that can explain computer science concepts in a **custom teaching style**, while demonstrating deep understanding of model behavior, limitations, and improvement strategies.

Scenario

You are hired by a university to develop an **AI-powered academic assistant** that helps students understand complex computer science concepts.

However, existing models such as ChatGPT or Claude often:

- Provide generic explanations
- Hallucinate incorrect information
- Fail to match the teaching style required in your course

Your task is to **fine-tune a small language model** to:

- Provide accurate explanations
- Follow a **consistent teaching style**
- Improve clarity and understanding for students

Important Instructions

- You **must NOT directly rely on AI-generated outputs without modification**
- All explanations, dataset creation, and reasoning must reflect **your own understanding**
- Marks will be awarded primarily for **reasoning, analysis, and decision-making**, not just code

Task Requirements

Part 1: Dataset Creation (Manual Work Required)

1. Use a dataset such as:
 - Stanford Question Answering Dataset
 - ELI5 Dataset

2. Select **20–30 samples** and **rewrite them manually** into instruction format:

Instruction: Explain recursion using a real-life analogy

Output: (Your own explanation)

Your explanations must be:

- Original
- Conceptually correct
- Written in your own words

Part 2: Teaching Style Definition

Define your **custom teaching style**, for example:

- Step-by-step explanation
- Analogy-based learning
- Beginner-friendly
- Mathematical/technical

Then **rewrite your dataset** to strictly follow this style.

Part 3: Model Selection

Choose a small language model such as:

- GPT-2
- DistilGPT2

Explain:

- Why you selected this model
- Its advantages and limitations

Part 4: Fine-Tuning Strategy

Choose ONE method:

- LoRA (Low-Rank Adaptation)
- Adapters
- Full fine-tuning

Explain:

- Why this method is suitable
- How it affects parameters and performance

Part 5: Model Evaluation (Critical Analysis)

Test your model using the same prompt:

Explain dynamic programming

Compare:

- Output from base model
- Output from fine-tuned model

Analyze:

- Improvement in clarity
- Reduction in hallucination
- Consistency in teaching style

Part 6: Failure Analysis

Identify **at least 2 failure cases**, such as:

- Incorrect explanation
- Oversimplification
- Hallucination

For each case, explain:

- Why the model failed
- Whether the issue is due to data, model, or training

Part 7: Manual Improvement

Fix ONE failure case by:

- Improving dataset quality OR
- Adding better examples OR
- Adjusting instruction format

Then evaluate whether the fix improves results.

Deliverables

Submit the following:

1. Curated dataset (manual)
2. Code implementation
3. Model outputs (before & after fine-tuning)
4. **Reflection report** including:
 - What the model learned
 - Where it failed
 - Key insights

Marking Criteria

Component	Weight
Dataset Creation	25%
Reasoning & Justification	25%
Experimentation	20%
Failure Analysis	20%
Code	10%

By completing this lab, you will understand:

- How LLMs behave beyond just usage
- The importance of **data quality and human input**
- Why fine-tuning is not just technical—but conceptual and analytical